DocketPilot : An Agentic AI System for Indian Legal Research, Drafting, Anti-Hallucination Verification and Case Management

A PROJECT REPORT

Submitted by

Suryaprakash P - 113023148050
Srigaran N - 113023148048
Selvaraj Vasanthakumar - 113023148046

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Vel Tech High Tech
Dr.Rangarajan Dr.Sakunthala Engineering College
Autonomous - Affiliated to Anna University
Avadi, Chennai

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

Vel Tech High Tech
Dr.Rangarajan Dr.Sakunthala Engineering College
Autonomous - Affiliated to Anna University
Avadi, Chennai

BONAFIDE CERTIFICATE

Certified that this project entitled **"DocketPilot : An Agentic AI System for Indian Legal Research, Drafting, Anti-Hallucination"** is the Bonafide work of "**Selvaraj Vasanthakumar, Srigaran N, Suryaprakash P"** who carried out the work under my supervision.

SIGNATURE

Dr.SARAVANAN R , M.E, Ph.D.,
SUPERVISOR
Professor
Department of CSE(Artificial Intelligence and Machine Learning)
Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College, Avadi, Chennai – 600 062.

SIGNATURE

Dr.MANOJ KUMAR D S, M.E,Ph.D.,
HEAD OF THE DEPARTMENT
Associate Professor
Department of CSE(Artificial Intelligence and Machine Learning)
Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College, Avadi, Chennai – 600 062.

Submitted for the ODD Semester Examination held on ……
at VEL TECH HIGH TECH Dr. RANGARAJAN Dr. SAKUNTHALA ENGINEERING COLLEGE, AVADI, CHENNAI – 600 062.

ACKNOWLEDGEMENT

We wish to express my sincere thanks to the people who extended their help and support during our **Project Phase I** work.

We wish to express my sincere thanks to all those who extended their help and support during our Project Phase I work. First of all, We would like to express my deep gratitude to our beloved and respected **Founder President and Chairman, Col. Prof. Vel. Shri. Dr. R. Rangarajan, B.E (EEE), B.E (Mech), M.S (Auto),** Foundress President and Vice Chairman, (Late)**Dr. Sagunthala Rangarajan, M.B.B.S.,** Chair Person and Managing Trustee, **Dr. Mrs. Mahalakshmi Rangarajan.K, B.E, M.B.A (UK), Ph.D.,** for their kind encouragement, valuable guidance, and continuous support throughout the project. Their constant motivation and inspiring leadership have been a great source of strength and confidence in successfully completing our work.

We would like to express sincere gratitude to our honourable **Principal, Prof. Dr.E. Kamalanaban, B.E, M.E, Ph.D.,** for his invaluable guidance, constant encouragement, and strategic insights throughout the course of this Project Phase I.

We also deeply grateful to our beloved **Dean Academics, Prof. Dr.V.R.Ravi**, **B.E., M.E., Ph.D.,** for extending his constant support and creating an environment that encourages our academics and research endeavors.

We are thankful and extremely grateful to our **HoD, Dr.Manoj Kumar D S, M.E., Ph.D.,** Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), for his sustained help, guidance, and inspiration in doing Project Phase I.

We would like to express special thanks to our **Project Coordinator Dr. R.Saravanan , M.E., Ph.D., Professor,** for his useful advice and suggestions for my Project Phase I completion.

We extend sincere thanks to our Project Phase I **Supervisor, <<Name of the Supervisor, Qualifications, Designation >>** for the constant guidance, timely advice, valuable insights, and encouragement throughout the course of this project.

Finally, we express our heartfelt gratitude to all our **Faculty members and Technical Staff** for their continuous support and to our **Parents** for their unwavering love, encouragement and moral support throughout this journey.

CERTIFICATE OF EVALUATION

College Name : Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College

Degree : B.E

Branch : CSE(Artificial Intelligence and Machine Learning)

Semester : VII

S.No	Name of the Student(s)	Title of the Project	Name, Designation & Department of the Supervisor	PO & PSO Mapping	SDG Mapping
1	Suryaprakash P	**DocketPilot : An Agentic AI System For Indian Legal Research , Drafting , Anti Hallucination , Verification and Case Management**	**Dr. Saravanan R, M.E., Ph.D. Professor , CSE(AI&ML)**	**PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PSO1, PSO2**	**SDG 4, SDG 9, SDG 11, SDG 12, and SDG 17**
2	Selvaraj Vasanthakumar				
3	Srigaran N				

The report of the **Project Phase I** work submitted by the above students in partial fulfilment for the award of degree, Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning) for the viva voce examination held at Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College on_____has been evaluated and confirmed to be reports of the work done by the above students

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

Chapter No.	TITLE	Page No.
	ABSTRACT	
	LIST OF TABLES	
	LIST OF FIGURES	
1	INTRODUCTION	1
2	LITERATURE SURVEY	4
3	METHODOLOGY	9
	3.1 MODEL ARCHITECTURE	9
	3.2 DATA COLLECTION AND PREPARATION	13
	3.3 PRIVACY GATEWAY AND DOCUMENT INTAKE	14
	3.4 MULTI-AGENT WORKFLOW	15
	3.5 HYBRID RETRIEVAL AND RE-RANKING	16
	3.6 CITATION VERIFICATION AND AUTO-CORRECTION	17
	3.7 EVALUATION METHODOLOGY	18
4	RESULTS AND DISCUSSION	19
	4.1 APPLICATION INTERFACE	19
	4.2 EFFECT OF EACH PIPELINE LAYER	21
	4.3 CITATION VERIFICATION ACCURACY	22
	4.4 RETRIEVAL AND PRIVACY RESULTS	23
	4.5 COST AND SPEED	24
	4.6 COMPARISON WITH EXISTING SYSTEMS	25
	4.7 EXPLAINABILITY WITH LIME	26
5	CONCLUSION AND FUTURE WORK	28
6	REFERENCES	29
7	APPENDIX	31

ABSTRACT

Large language models can draft legal text fluently, but they also invent judgments, misquote passages and cite provisions that are no longer in force, and they normally require client papers to be sent to a hosted model. This project presents DocketPilot, an agentic AI system for Indian legal research, drafting, anti-hallucination verification and case management. Seven agents coordinated by a LangGraph state graph analyse case papers, retrieve statutes and judgments, draft documents with a source tag on every legal statement, verify and repair every citation, list risks and missing facts, and derive deadlines for a hearing calendar. A local privacy gateway masks names and identity numbers before any text reaches the model, and the real names are restored only after the lawyer approves the output. The knowledge base holds 22,767 Supreme Court and High Court judgments with full text, 79 Central Acts and 503,701 searchable passages. On a benchmark of 756 corrupted and 156 genuine citations the verifier reached 99.6% recall and 99.3% precision. With the same model, the full pipeline reduced unreliable citations from 16.1% to 0.0% and cut the cost of a verified answer by 87.2% compared with sending whole judgments to the model.

Keywords — *Large Language Models (LLMs), Agentic AI, Retrieval-Augmented Generation (RAG), Legal Hallucination, Citation Verification, LangGraph, Privacy-Preserving NLP, Indian Legal Informatics, Explainable AI (LIME)*

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Feature comparison of DocketPilot with related systems	8
3.1	Technology stack	13
3.2	Contents of the knowledge base	14
3.3	Agents of the LangGraph workflow	15
4.1	Comparison with published Indian legal-AI systems	26

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Block diagram of DocketPilot	10
3.2	End-to-end workflow of DocketPilot	12
3.3	Composition of the knowledge base by court	14
3.4	Outcomes of automatic correction across the evaluation answers	17
4.1	Sign-in page	19
4.2	Masking review before upload	19
4.3	Verified research answer	20
4.4	Draft with verification panel	20
4.5	Hearing calendar	20
4.6	Statute finder	20
4.7	Case dashboard	21
4.8	Unreliable citations as each layer of the pipeline is added	22
4.9	Faulty citations caught by type	23
4.10	Confusion matrix of the verifier	23
4.11	F1 of citation-error detection	23
4.12	Unreliable citations compared	23
4.13	Retrieval methods compared	24
4.14	Speed and accuracy of search	24
4.15	Recall of the privacy gateway by type of identifier	24
4.16	Cost reduction against an AI baseline	25
4.17	Cost per 1,000 research answers	25
4.18	Time per answer by stage	25
4.19	Tokens per answer	25
4.20	DocketPilot against published Indian legal-AI systems	26
4.21	LIME explanation of the evidence re-ranker	27

CHAPTER 1
INTRODUCTION

Artificial Intelligence (AI) is increasingly used in professional fields to search information, analyse documents and support decision-making. In legal practice, advocates read statutes, court judgments, case files, contracts and notices before they prepare arguments or advise clients. AI tools can help with legal research, summarisation and first drafts, and so reduce repetitive work. Legal work, however, demands accuracy, traceable sources, confidentiality and professional judgment. A legal AI system must therefore do more than generate fluent text: every statement it makes about the law must be checkable.

A major weakness of generative AI is hallucination, where a model produces convincing but incorrect or unsupported content. In law this takes the form of judgments that do not exist, real judgments cited for a point they never decided, wrong page or year numbers, misquoted passages and sections that have been repealed or renumbered. Studies of general models have measured hallucination rates between 69% and 88% on verifiable questions about court cases, and even commercial research tools that use retrieval still produce misgrounded answers in 17% to 33% of cases. Such errors can mislead a court and expose the advocate to professional consequences, so the output of a model cannot be trusted without an independent check against the original sources.

The problem is sharper in India. Research involves Central Acts and their amendments, the move from the Indian Penal Code, Code of Criminal Procedure and Evidence Act to the Bharatiya Nyaya Sanhita, Bharatiya Nagarik Suraksha Sanhita and Bharatiya Sakshya Adhiniyam in 2024, and judgments of the Supreme Court and many High Courts reported in several citation formats (SCC, SCR, AIR and neutral citations). The provision that applies depends on the date of the offence or the cause of action. A useful assistant must retrieve material from identifiable sources, keep the date on which a provision was in force, and confirm every citation against the judgment itself before it shows a result to the lawyer.

Confidentiality adds a second constraint. Case papers carry names, Aadhaar and PAN numbers, bank accounts, telephone numbers and addresses. Sending these to a hosted model would breach the duty of confidence that an advocate owes a client. Any practical system must keep identities on the lawyer's own machine and give the model only masked text.

DocketPilot: An Agentic AI System for Indian Legal Research, Drafting, Anti-Hallucination Verification and Case Management addresses these problems with a coordinated multi-agent architecture. A LangGraph state graph runs seven specialised agents for orchestration, document analysis, research, drafting, verification, strategy and risk, and deadline monitoring, and it pauses for the advocate's approval before any document is released. A single conversational interface routes ordinary questions to verified research answers, similar-case search, verification of pasted text and reminders, while drafting requests start the full agent graph.

The knowledge base is built offline from the open Supreme Court and High Court judgment collections published under the CC-BY-4.0 licence and from Central Acts published on India Code. Judgments are cleaned, split into headnote and body, cut into passages and indexed twice: in PostgreSQL for keyword search and in the Qdrant vector database for meaning-based search. Retrieval-Augmented Generation (RAG) supplies the most relevant passages to the language model, gpt-oss-120b served through Groq, which is the only generative model in the system.

Before any text reaches the model, a privacy gateway replaces identities with placeholders such as PERSON_1 and ORG_2. It uses rule patterns with checksums for structured identifiers and a named-entity model for names. The advocate reviews the masked text, and the mapping from placeholder to real name is kept in an encrypted vault on the local machine. After generation, a verification engine extracts every citation, quotation and statutory reference, resolves it against the case store, checks quotations word for word, asks a separate judge prompt whether the cited passage supports the sentence, and confirms that each section was in force on the relevant date. Automatic correction then repairs wrong citation strings and removes claims that cannot be grounded, so the lawyer receives only checked text.

For case management, DocketPilot stores each matter as a case file with its papers, facts, drafts and an audit log. Dates found in the papers are filed to a hearing calendar, each anchored to the sentence it came from, and approved drafts are exported as Word documents with the real names restored. The system runs locally with Docker, uses role-based accounts for advocates and juniors, and was evaluated on citation-error detection, retrieval quality, privacy masking, answer reliability, cost and speed. The objective is a reliable, verifiable and privacy-

conscious assistant for Indian legal practice, in which the advocate remains responsible for every document that is filed.

CHAPTER 2
LITERATURE SURVEY

The following studies were selected from academic and research sources relevant to legal hallucinations, reliability of AI legal research tools, judicial AI applications, Indian legal assistants and automated detection of unsupported legal content. The techniques and findings below summarise the cited publications.

S. No.	Title of the Paper	Name of the Journal / Source	Year	Algorithms / Techniques	Findings
1	Neethi AI: An AI-Powered Multi-Agent System for Indian Legal Advisory	IEEE Conference Publication	2026	Multi-agent architecture using CrewAI; hybrid Retrieval-Augmented Generation (RAG) with BGE-M3 dense and BM25 sparse retrieval; Qdrant vector search; citation verification; multilingual voice interaction.	The paper describes a legal assistant designed for the Indian legal ecosystem. It reports low latency for common queries, citation checking, drafting support, and voice interaction in 12 Indian languages. Its stated focus is common citizens rather than lawyers and that personal-information security is a limitation.
2	LegalMind: Agentic AI-Driven Process Optimization and Cost Reduction in Legal	IEEE Access	2025	Agentic AI framework using fine-tuned DeepSeek R1 and a structured legal dataset for legal workflow automation.	The paper reports a 42.6% reduction in cost and a 60.8% improvement in document-processing speed compared with

S. No.	Title of the Paper	Name of the Journal / Source	Year	Algorithms / Techniques	Findings
	Services Using DeepSeek				baseline AI models. These results suggest that agentic AI may streamline repetitive legal tasks and reduce operational effort. This is relevant to DocketPilot's goal of automating legal workflows while keeping outputs reviewable.
3	Artificial intelligence in the judiciary: a systematic review	The Judicial Review	2026	Systematic literature review of AI use in judicial settings, including case handling, decision support, and workflow automation.	The review finds that AI is increasingly used to improve court efficiency through case handling, decision support, and workflow automation. It also highlights trust, transparency, and human oversight as important conditions for adoption.
4	Generative AI in Legal Practice: Applications, Challenges,	SSRN	2025	Review of LLM-based legal applications, including legal	The review discusses potential productivity improvements in

S. No.	Title of the Paper	Name of the Journal / Source	Year	Algorithms / Techniques	Findings
	and Ethical Considerations			research, drafting, contract analysis, and client communication.	legal research, drafting, contract analysis, and client communication. It also identifies hallucinations, privacy concerns, and professional responsibility as important challenges.
5	Artificial Intelligence and the Practice of Law Part 1: Lawyers Must be Professional and Responsible Supervisors of AI	SSRN / Kentucky Bench & Bar	2023	Conceptual analysis of AI-assisted legal work, including document review, summarization, translation, and legal analysis.	The article argues that AI can improve efficiency but is not infallible. Lawyers should supervise AI-generated work, verify sources and documents, and protect client confidentiality. Its central message is that AI should support professional judgment rather than replace the lawyer's responsibility for the final work.
6	Hallucination-Free? Assessing the Reliability of	Journal of Empirical Legal Studies	2025	Pre-registered evaluation of retrieval-augmented	Retrieval reduces but does not remove hallucination:

S. No.	Title of the Paper	Name of the Journal / Source	Year	Algorithms / Techniques	Findings
	Leading AI Legal Research Tools			commercial tools (Lexis+ AI, Westlaw AI-Assisted Research, Ask Practical Law AI) with human grading of answers.	17% of Lexis+ AI and 33% of Westlaw answers were incorrect or misgrounded. Citing a real case for the wrong proposition was a frequent failure, which motivates checking support and not only existence.
7	Who Checks the Citations? Benchmarking Legal Hallucination Detection	arXiv preprint	2026	Benchmark of citation-error detection on excerpts of legal briefs; models used as agentic checkers with search tools.	The best agentic configuration caught 84.4% of bad citations and the best F1 was 68.8%. The same open model used by DocketPilot (gpt-oss-120b) caught 55.1% when used alone, so a checker grounded in a complete case store is needed.
8	NyayaAI: An AI-Powered Legal Assistant Using Multi-Agent Architecture and Retrieval-	arXiv preprint	2026	Multi-agent orchestration with a domain classifier, FAISS semantic retrieval over Indian statutes and case law, and a	Reports 74% retrieval precision, 72% response accuracy and an F1 of 0.71. It does not verify the citations it emits or mask

S. No.	Title of the Paper	Name of the Journal / Source	Year	Algorithms / Techniques	Findings
	Augmented Generation			compliance check before delivery.	client data, and the test-set size is not reported.

The survey shows three gaps. First, retrieval alone does not make legal answers reliable: commercial retrieval tools still misground a sixth to a third of their answers, and the most common failure is a real authority cited for a point it does not support. Second, existing Indian assistants such as Neethi AI and NyayaAI focus on retrieval, routing and accessibility, and none of them verifies each citation against the judgment text or repairs the output automatically. Third, client confidentiality is treated as a limitation rather than designed into the pipeline. DocketPilot is built to close these gaps by combining retrieval, source-tagged drafting, citation verification with automatic correction, and local privacy masking in one application for Indian lawyers.

Table 2.1 Feature comparison of DocketPilot with related systems

System	Users	Retrieval	Citation check	Auto-repair	PII masking	Calendar
DocketPilot	Lawyers	Hybrid + re-rank	Yes	Yes	Local, encrypted	Yes
Neethi AI	Citizens	BGE-M3 + BM25	Yes	No	Listed as a limitation	No
NyayaAI	General	FAISS dense	No	No	No	No
LegalMind	Firms	Not reported	No	No	No	No
Lexis+ AI	Lawyers (US)	Proprietary	No	No	No	No

CHAPTER 3
METHODOLOGY

The methodology follows the path of a request through the system. Section 3.1 presents the overall architecture. Sections 3.2 to 3.6 describe each stage in the order in which it acts on the data: building the knowledge base, masking client documents, running the agent graph, retrieving evidence, and verifying and correcting the output. Section 3.7 explains how the system was evaluated.

3.1 MODEL ARCHITECTURE

DocketPilot is organised in five layers (Fig. 3.1). The user layer is a single-page web application through which an advocate or junior signs in, attaches case papers, reviews the masking, asks questions, reads verified results and approves drafts. The backend layer is a FastAPI service written in Python. It authenticates users with JSON Web Tokens, keeps every firm's data separate, writes an append-only audit log, and hosts the privacy gateway and a rule-based chat router that decides which tool should answer a message.

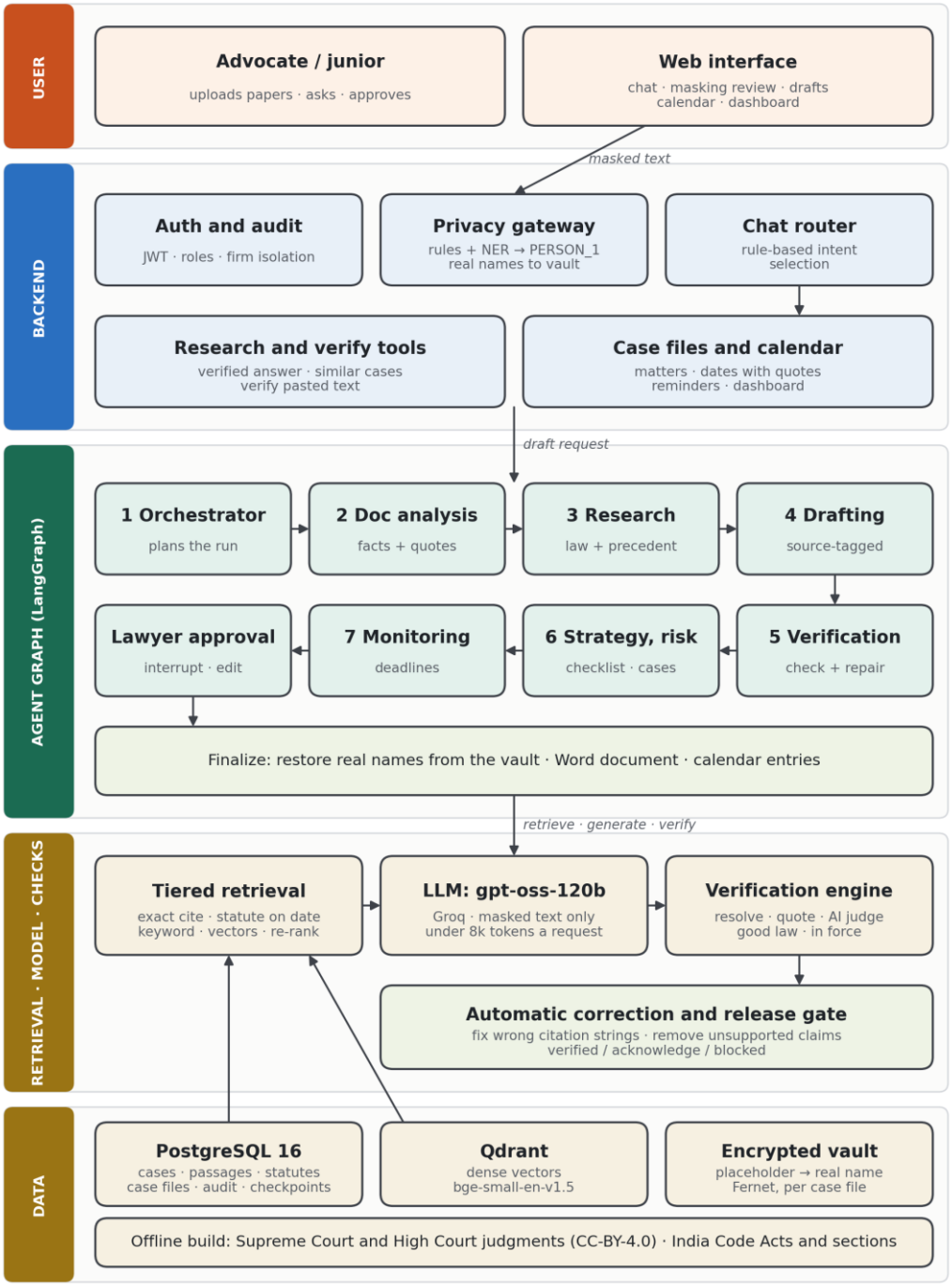


Fig. 3.1 Block diagram of DocketPilot

The agent layer is a LangGraph state graph with seven agents. Its state is saved to PostgreSQL after every step, so a run can resume after a crash or a restart, and it pauses at an approval interrupt so that nothing leaves the system without the advocate's consent. The retrieval and model layer contains the tiered retriever, the language model gpt-oss-120b served by Groq, and the verification engine. The data layer holds PostgreSQL 16 (cases, passages, statutes, case files, audit records and graph checkpoints), the Qdrant vector database, and the encrypted vault that maps placeholders to real names.

Fig. 3.2 shows the complete workflow across the lanes: the lawyer's actions, the frontend screens, the backend services, the LangGraph agents, the verification steps, and the data and models each step uses. The following sections describe the stages in the order in which a request passes through them.

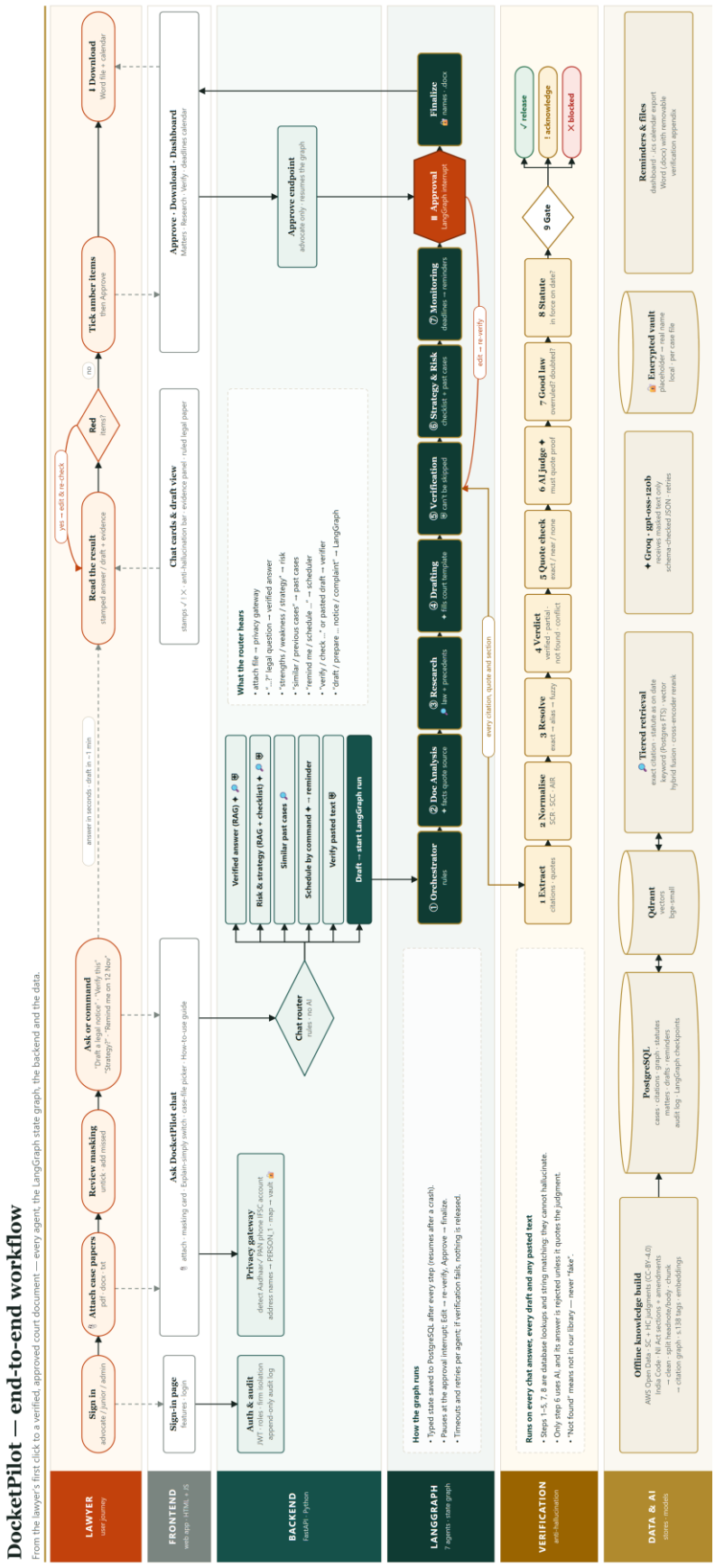


Fig. 3.2 End-to-end workflow of DocketPilot

Table 3.1 Technology stack

Layer	Component	Purpose
Frontend	HTML, CSS, JavaScript (single page)	Chat, masking review, drafts, calendar, dashboard
Backend	FastAPI, Python 3.11	REST API, authentication, routing, audit log
Agents	LangGraph	Seven-agent state graph with checkpoints and approval interrupt
Language model	gpt-oss-120b on Groq	Answers, drafts and support judgments on masked text
Relational store	PostgreSQL 16	Cases, passages, statutes, full-text search, case files
Vector store	Qdrant 1.12	Dense passage vectors (bge-small-en-v1.5, 384 dimensions)
Re-ranker	MiniLM cross-encoder	Re-orders retrieved passages by relevance
Privacy	Regex with checksums, spaCy NER, Fernet	Masking, review and encrypted placeholder vault
Deployment	Docker Compose	PostgreSQL, Qdrant, Redis and object store on one machine

3.2 DATA COLLECTION AND PREPARATION

The knowledge base is built once, offline, from public sources. Supreme Court judgments and High Court judgments of five benches (Kerala, Karnataka, Telangana, Madras and its Madurai bench) are taken from the open Indian judgment collections on the AWS Registry of Open Data, which are released under the CC-BY-4.0 licence. Central Acts, their sections and their amendment footnotes are taken from India Code, published by the Legislative Department of the Government of India.

Each judgment passes through a fixed preparation pipeline. The text is cleaned of page furniture, and the headnote written by the law reporter is separated from the body of the judgment. The headnote is stored as its own segment and is never used as evidence, because it is not the court's own words. The body is cut into overlapping passages, each tagged with its page so that a citation can point to the exact place. Case names, dates, courts and every reported citation (neutral, SCR, SCC and AIR) are normalised into a single case record so that any form of a citation can be resolved to the same judgment. Citations between judgments are extracted into a citation graph that is later used to check whether a precedent has been overruled or doubted.

Statutes are stored section by section with the dates from which each version was in force, so the system can answer what a section said on a given date. Every rule file that encodes a legal deadline or limitation quotes the statute, and an automatic test checks that the quoted words really occur in the India Code text. The resulting store holds 52,166 cases for citation resolution, 22,767 judgments with full text, 79 Central Acts with 8,558 sections, and 583,931 passages, of which 503,701 are embedded for vector search (Fig. 3.3).

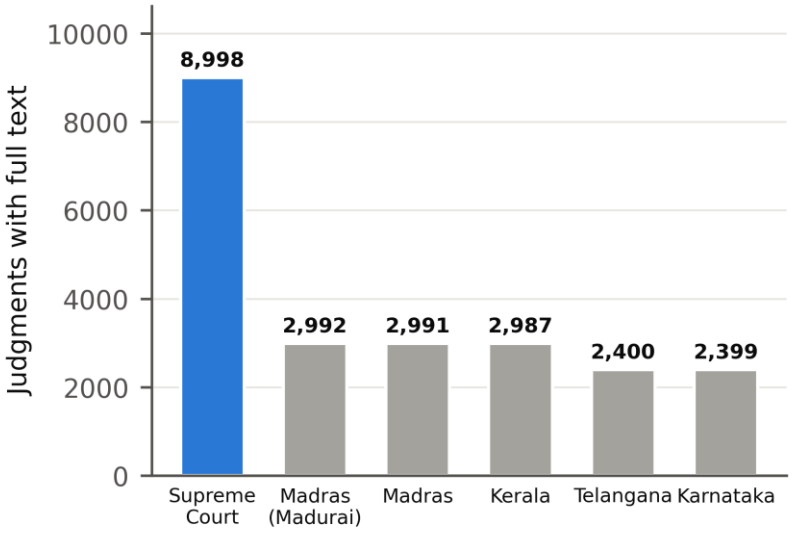


Fig. 3.3 Composition of the knowledge base by court

Table 3.2 Contents of the knowledge base

Item	Count	Source
Cases available for citation resolution	52,166	Supreme Court 1950-2026, High Courts 2022-2025
Judgments with full text	22,767	AWS Open Data (CC-BY-4.0)
Central Acts / sections	79 / 8,558	India Code
Passages (keyword index)	583,931	PostgreSQL full-text search
Passages with vectors	503,701	Qdrant, bge-small-en-v1.5

3.3 PRIVACY GATEWAY AND DOCUMENT INTAKE

Client papers enter the system only through the privacy gateway. An uploaded PDF, Word or text file is converted to plain text and scanned by two recognisers. The first uses rule patterns with checksums for structured identifiers: Aadhaar numbers (validated with the Verhoeff checksum), PAN, bank account and IFSC codes, telephone, vehicle registration numbers, e-mail addresses and postal addresses. The second is a spaCy named-entity model that finds the names of people and organisations. Every value found is replaced by a consistent

placeholder, so that the same person becomes PERSON_1 everywhere in the case file and the model can still reason about who did what.

The masked text is shown to the advocate before it is stored, with each placeholder highlighted. The advocate can untick a mask that is not needed or mark a name the recogniser missed. Only after this review is the masked text saved and made available to the agents. The mapping from placeholder to real value is encrypted with Fernet (AES in CBC mode with an HMAC) under a key held in the local environment file, and it is stored per case file. Real names are restored only on the advocate's screen and in the approved Word document, so the language model never receives an identity.

When the papers are analysed, the document-analysis agent extracts facts such as the parties, the cheque or agreement details and the important dates, and it keeps a fact only if the sentence that supports it is found in the papers. Each date is filed to the case calendar together with that quote, so the lawyer can see where every reminder came from.

3.4 MULTI-AGENT WORKFLOW

Drafting requests start the LangGraph workflow, in which each agent is a node that reads and writes a typed shared state. The agents and their responsibilities are listed in Table 3.3. The graph runs them in order, with time limits and retries for each node, and it saves the state after every step.

Table 3.3 Agents of the LangGraph workflow

Agent	Input	Output
1 Orchestrator	Request and case file	Workflow plan, document type and court
2 Document analysis	Masked papers	Facts, each with a supporting quote
3 Research	Facts and issues	Statute sections in force and re-ranked judgment passages
4 Drafting	Facts and sources	Draft with a source tag on every legal statement
5 Verification	Draft	Checked and repaired draft with a verdict for each citation
6 Strategy and risk	Facts and draft	Missing facts, weak points and similar past cases
7 Monitoring	Facts and statutes	Statutory deadlines and reminders
Approval and finalize	Advocate's decision	Released Word document with real names restored

The drafting agent never states law without a source. It writes tags such as [P1] or [S2] after each legal statement, which refer to the retrieved passage or statute section that supports it, and the system replaces each tag with the proper citation afterwards. Because the model only chooses among sources that were actually retrieved, it cannot invent a citation string. The verification agent cannot be skipped: if it fails or times out, nothing is released.

After monitoring, the graph stops at an approval interrupt. The advocate reads the draft beside its evidence panel, ticks any item flagged for acknowledgement, edits the text if needed and approves it. An edit sends the draft back through verification. On approval, the finalize step restores the real names from the vault, produces the Word file and adds the deadlines to the calendar. Questions that do not need a full draft, such as a research question, a request for similar cases or a request to verify pasted text, are answered directly by the chat router using the same retrieval and verification components.

3.5 HYBRID RETRIEVAL AND RE-RANKING

Retrieval proceeds from exact to fuzzy so that the cheapest reliable method is used first. An exact citation in the question is looked up directly in the case store, and a reference to a section is answered from the statute table as on the relevant date. For open questions, two searches run in parallel. PostgreSQL full-text search ranks passages by keyword overlap, and dense search in Qdrant finds passages with similar meaning, using 384-dimensional bge-small-en-v1.5 embeddings stored as 8-bit integers to save memory.

The two ranked lists are combined by reciprocal rank fusion, which adds 1/(k + rank) for every list in which a passage appears, so a passage that ranks well in both lists rises to the top. The fused candidates are then re-scored by a MiniLM cross-encoder that reads the question and the passage together. Finally, the retrieved text is fitted to a fixed token budget so that a single request to Groq stays within the provider's limit of 8,000 tokens per request on the free tier. Every passage keeps its case, page and paragraph so that the final citation can point to the exact place.

Requests to the model are sent through a client that rotates among several API keys, retries on rate limits and stops cleanly when the daily quota is reached. Only masked text is ever sent, and the model is asked for structured JSON whose fields are validated before use.

3.6 CITATION VERIFICATION AND AUTO-CORRECTION

Every chat answer, every draft and any text pasted by the user passes through the same verification engine. Its steps are deterministic database lookups and string comparisons, with a single step that uses the language model. First, every citation, quotation and statutory reference is extracted. Citations are normalised (for example, "(2021) 5 SCC 283" and "2021 INSC 529") and resolved against the case store by exact key, then by alias, then by name and year. Each citation receives a verdict: verified, partial, conflict (the details point to different judgments) or not found.

Quotations are compared with the judgment text and marked exact, near or absent. For each citation that exists, a separate judge prompt is shown the cited passage and the sentence that relies on it, and it must answer whether the passage supports the sentence, quoting the passage as proof; an answer without a matching quote is rejected. The citation graph is then consulted to see whether the judgment has been overruled or doubted by later cases, and every statutory reference is checked to confirm that the section exists and was in force on the relevant date.

Automatic correction acts on the verdicts. When the named case exists but the citation string is wrong, the correct citation replaces it. When a citation cannot be grounded, or the judge finds that the passage does not support the sentence, the citation is removed together with the sentence that relied on it, and case names written without any citation are removed. The release gate then marks the output as verified, as needing acknowledgement, or as blocked. The user sees only the repaired text and a one-line summary such as "10 references checked, 4 removed" (Fig. 3.4).

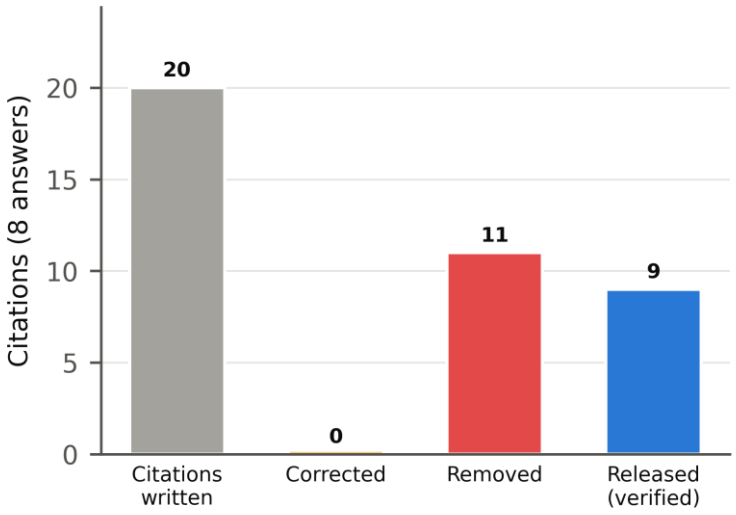


Fig. 3.4 Outcomes of automatic correction across the evaluation answers

3.7 EVALUATION METHODOLOGY

All experiments use one model, gpt-oss-120b on Groq, so that differences come from the pipeline rather than from the model. Costs are computed from the token counts returned by the API at Groq list prices (US$0.15 per million input tokens and US$0.60 per million output tokens). Six evaluations were carried out.

- Citation-error detection: 150 real Supreme Court citations, each corrupted in five ways (wrong page, wrong year, swapped party names, wrong court and an invented case), plus citations used for the wrong proposition, giving 756 faulty and 156 genuine citations.
- Retrieval: 93 queries in which the headnote of a judgment must retrieve the body of the same judgment, measured by Recall@1, Recall@5 and mean reciprocal rank (MRR@10).
- Privacy masking: 1200 identifiers planted in held-out synthetic Indian case papers, measured by recall and leakage.
- Pipeline ablation: eight research questions across criminal, family, contract, property, arbitration and consumer law answered by the model alone, with retrieval, with source tags, and with the full pipeline.
- Cost and speed: the same questions answered by DocketPilot and by the same model reading the full judgments, and eight real High Court judgments processed as case papers.
- Explainability: LIME applied to the passage re-ranker for two questions.

A citation is counted as fabricated only if its format is one that the store covers completely (Supreme Court neutral and SCR citations). Citations in other formats are reported as not checkable rather than as fabricated, because a missing entry in an incomplete store does not prove that a judgment does not exist.

CHAPTER 4
RESULTS AND DISCUSSION

This chapter first shows the working application and then reports the measured results of the evaluations described in Section 3.7.

4.1 APPLICATION INTERFACE

The screens below were captured from the running system with synthetic case papers. Fig. 4.1 shows the sign-in page, which supports e-mail accounts and Google sign-in. After a file is attached, the masking review (Fig. 4.2) lists every identifier that will be hidden from the model, and the advocate can untick or add masks before the text is stored.

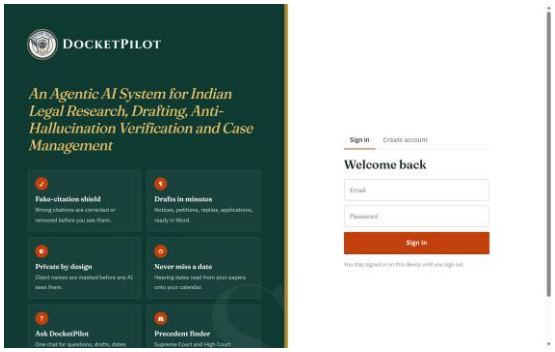


Fig. 4.1 Sign-in page

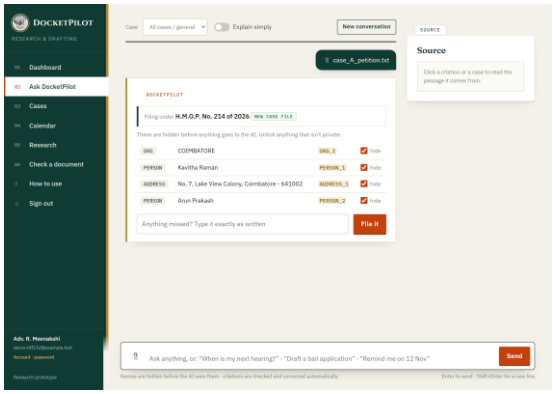


Fig. 4.2 Masking review before upload

Fig. 4.3 shows a research answer on divorce on the ground of cruelty and the wife's claim to maintenance. Each statement carries a citation that has been checked, and the summary line reports that ten references were checked and four were removed because their passages did not support the sentence. Fig. 4.4 shows a petition drafted by the agent graph with its verification panel, which states that the draft is clear to approve and lists the facts that are still needed before filing.

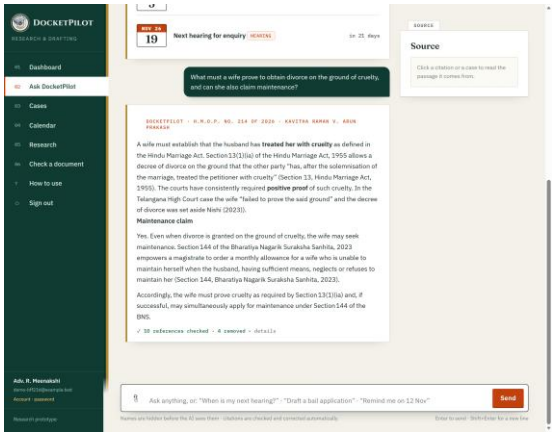


Fig. 4.3 Verified research answer

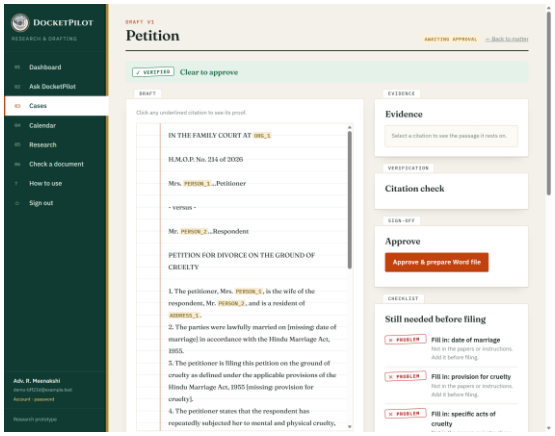


Fig. 4.4 Draft with verification panel

Dates found in the papers are filed to the hearing calendar (Fig. 4.5), each with the quote it came from. The statute finder (Fig. 4.6) searches the Central Acts by topic and shows the text of each section, and the dashboard (Fig. 4.7) lists the advocate's case files, drafts awaiting approval and upcoming deadlines.

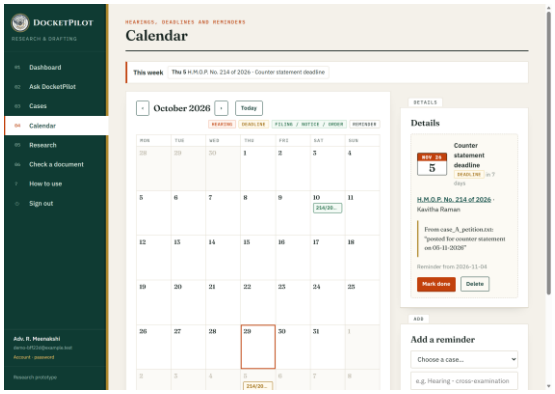


Fig. 4.5 Hearing calendar

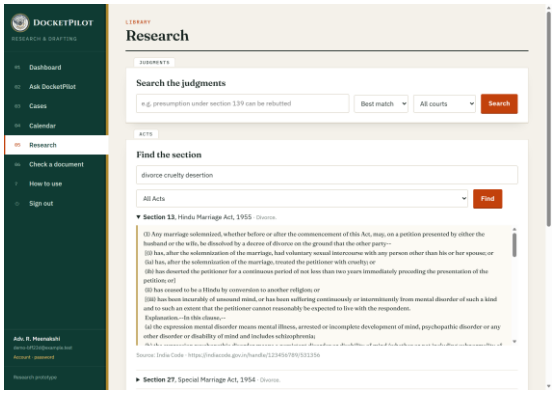


Fig. 4.6 Statute finder

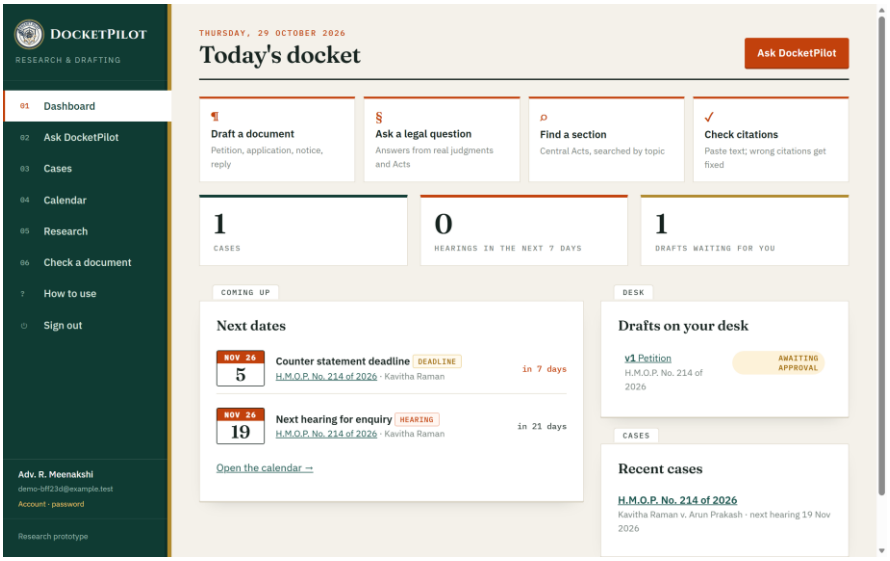


Fig. 4.7 Case dashboard

4.2 EFFECT OF EACH PIPELINE LAYER

Fig. 4.8 adds one layer at a time to the same model. Answering from memory (A), 11.3% of checkable citations were fabricated and 4.8% named real cases that do not support the stated point. Supplying retrieved passages (B) lowered fabrication to 7.7% but raised misattribution to 15.4%: given real material, the model attached it to claims it does not make. Requiring a source tag on every statement (C) removed fabrication completely, yet the judge rejected 38.9% of tagged citations as not supporting the exact sentence. Only verification with automatic correction (D) removed both failures: of 20 citations written in the full answers, 11 were removed with the claims that relied on them, and all 9 released citations were verified.

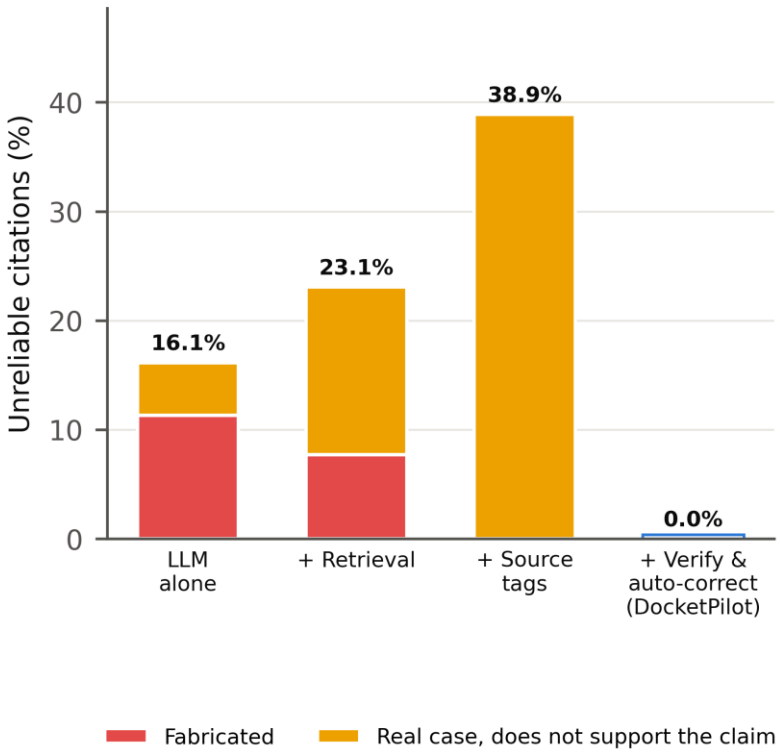


Fig. 4.8 Unreliable citations as each layer of the pipeline is added

The layers are therefore complementary rather than cumulative. Source tagging solves the problem of existence, because the model can only cite what was retrieved, while verification solves the problem of support. With eight questions the percentages are indicative; the pattern, not the exact values, is the main finding.

4.3 CITATION VERIFICATION ACCURACY

On the citation benchmark the verifier reached 99.6% recall, 99.3% precision and an F1 score of 99.5%. Invented cases, wrong courts, wrong years and wrong pages were all caught (Fig. 4.9). Swapped party names were the hardest corruption, with 98% caught, because the rest of the citation is correct. Among genuine citations, 2.7% were flagged, mostly as partial matches that ask the lawyer to look again rather than blocking the citation. Fig. 4.10 gives the confusion matrix.

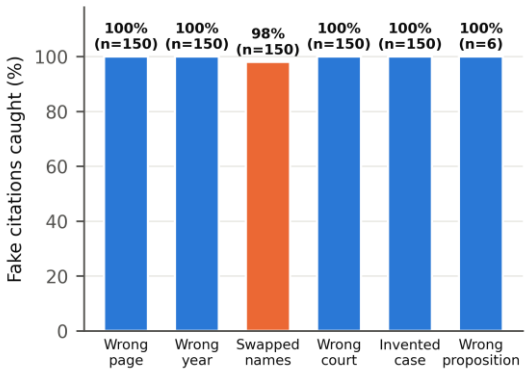


Fig. 4.9 Faulty citations caught by type

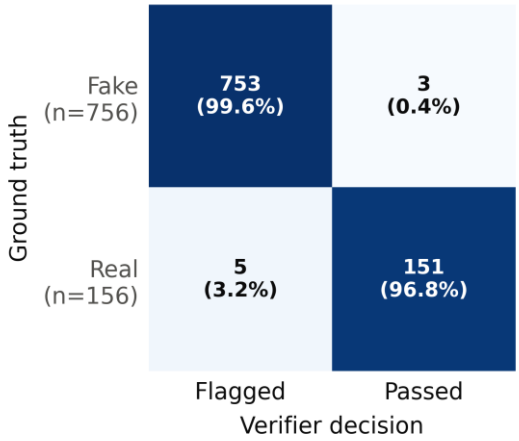


Fig. 4.10 Confusion matrix of the verifier

Fig. 4.11 compares the F1 score with published citation checkers. The same base model used as an agentic checker in a recent benchmark reached 55.1% recall and 30.5% F1, so the improvement comes from grounding the check in a complete case store and not from the model itself. Fig. 4.12 places DocketPilot's released answers beside published hallucination rates; the figures for other systems come from different jurisdictions and protocols and are given for context.

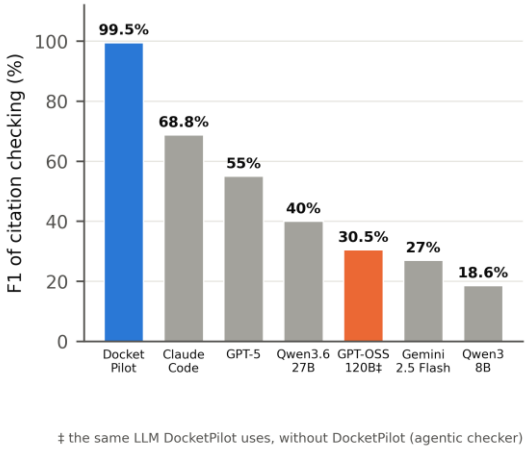


Fig. 4.11 F1 of citation-error detection

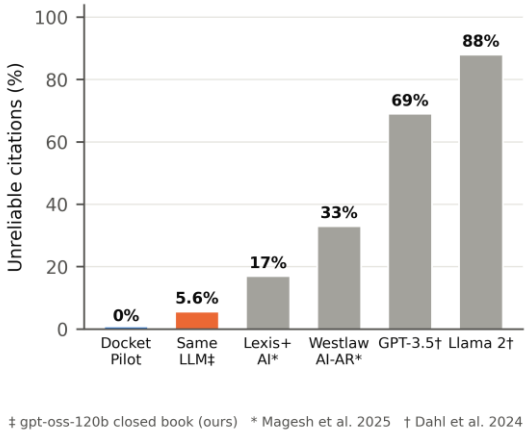
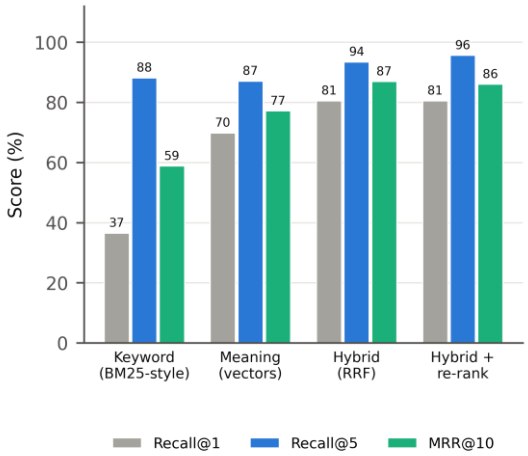


Fig. 4.12 Unreliable citations compared

4.4 RETRIEVAL AND PRIVACY RESULTS

Fig. 4.13 compares the retrieval methods. Keyword search alone found the right judgment in the top five for 88.2% of queries and vector search for 87.1%; fusing the two raised Recall@5 to 93.5%, and re-ranking raised it to 95.7% with an MRR of 0.861. Re-ranking costs time on a laptop CPU (Fig. 4.14), but exact citation

lookups take 1.9 ms and statute look-ups 3.1 ms at the 95th percentile, well inside the 50 ms target.



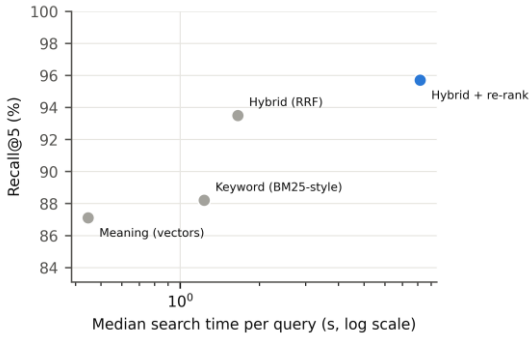


Fig. 4.13 Retrieval methods compared

Fig. 4.14 Speed and accuracy of search

The privacy gateway masked every structured identifier in the held-out papers: Aadhaar, PAN, bank, IFSC and telephone numbers all reached 100% recall (Fig. 4.15). Overall recall was 96.8%; the misses were person names, which the review screen is designed to catch before anything is stored. Precision was 81.2%, meaning some extra words were masked, which is the safe direction for confidentiality.

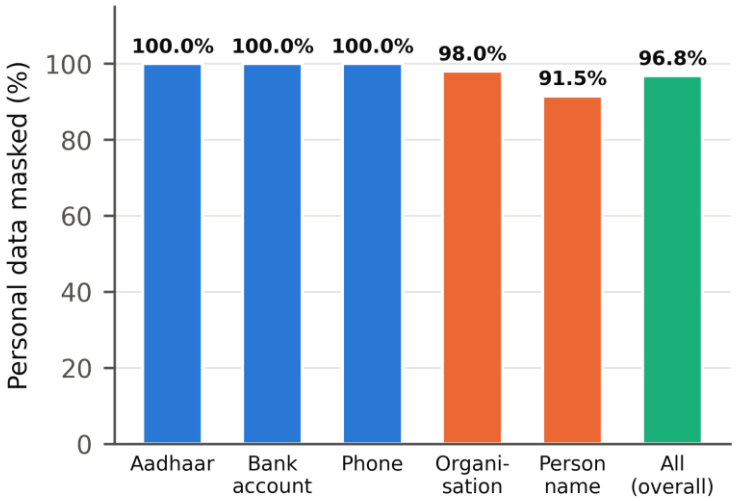


Fig. 4.15 Recall of the privacy gateway by type of identifier

4.5 COST AND SPEED

Because only the best passages are sent, a verified research answer cost US$0.00285 against US$0.02215 when the same model read the full judgments, a reduction of 87.2% that already includes every verification call (Fig. 4.16). This

reduction is larger than the 25% to 67.5% reported for structured navigation and embedding retrieval and the 42.6% reported for LegalMind. Fig. 4.17 shows the cost of one thousand answers.

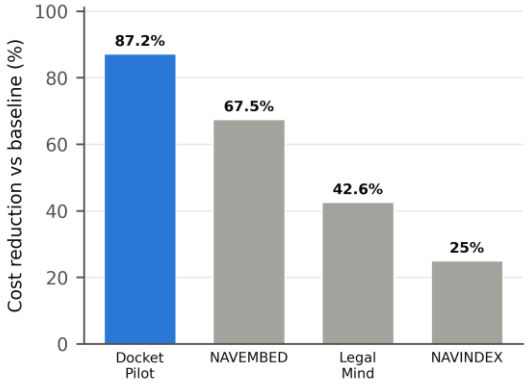


Fig. 4.16 Cost reduction against an AI baseline

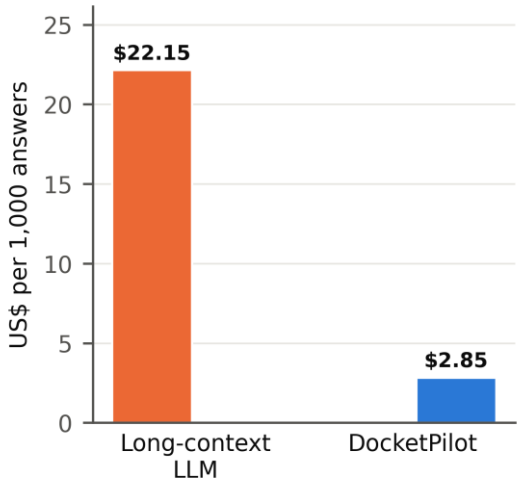


Fig. 4.17 Cost per 1,000 research answers

Verification is not free. In the ablation run, retrieval took 9.3 s per answer on a laptop CPU, generation 2.7 s and verification 2.2 s (Fig. 4.18), and the judge prompts used almost as many tokens as the answer itself (Fig. 4.19). Reading a case paper took 3.0 s against 4.7 s for the unassisted model, a saving of 35.9%. DocketPilot trades some raw speed for checking every statement.

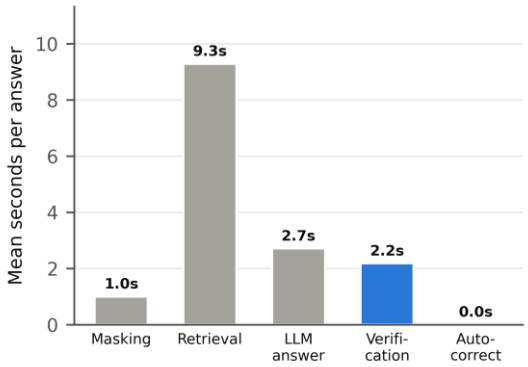


Fig. 4.18 Time per answer by stage

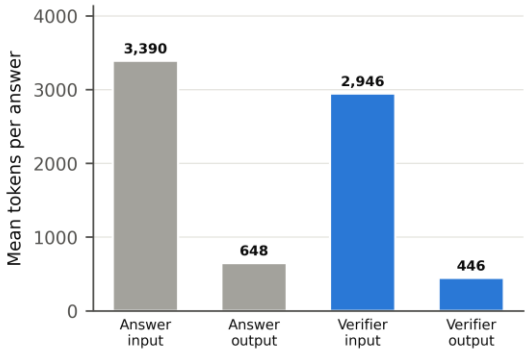


Fig. 4.19 Tokens per answer

4.6 COMPARISON WITH EXISTING SYSTEMS

Table 4.1 sets DocketPilot beside the Indian systems that report comparable scores, and Fig. 4.20 shows the same comparison. The first passage returned is the correct judgment in 80.6% of queries, above the 74% retrieval precision reported by NyayaAI. The ranking quality (MRR 0.861) is well above the mean

average precision of 0.36 reported by AQgR; when each query has one correct judgment, the two measures coincide. The verifier's F1 of 99.5% is above NyayaAI's headline F1 of 0.71. Each system was evaluated on its own test set, so the comparison positions the systems rather than ranking them. Neethi AI reports qualitative results without comparable numbers and is therefore compared by features in Table 2.1.

Table 4.1 Comparison with published Indian legal-AI systems

Measure	Other system	Reported	DocketPilot
Retrieval precision (top result)	NyayaAI	74%	80.6%
Headline F1	NyayaAI	0.71	0.995
Ranking quality (MAP / MRR)	AQgR	0.36	0.861
Cost reduction vs. full documents	LegalMind	42.6%	87.2%
Fabricated citations in answers	Same model alone	5.6%	0.0%

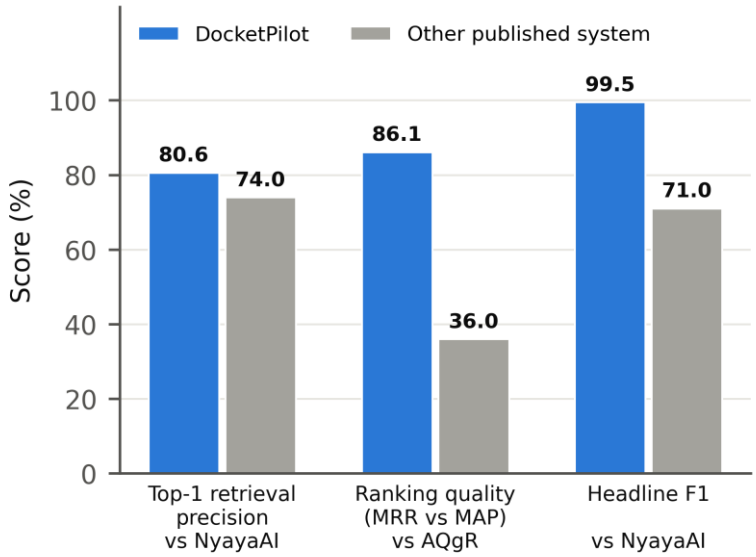


Fig. 4.20 DocketPilot against published Indian legal-AI systems

4.7 EXPLAINABILITY WITH LIME

To make the choice of evidence inspectable, LIME (Local Interpretable Model-agnostic Explanations) was applied to the passage re-ranker. Words of the selected passage were removed at random 400 times, each variant was scored by the re-ranker, and a weighted linear model was fitted to the scores; its coefficients show how much each word contributed. For a question on the presumption under

Section 139 of the Negotiable Instruments Act, the strongest words were the section number, "rebuttable", "negotiable" and "accused". For a maintenance question they were "wife", "husband", "claim" and "maintenance" (Fig. 4.21).

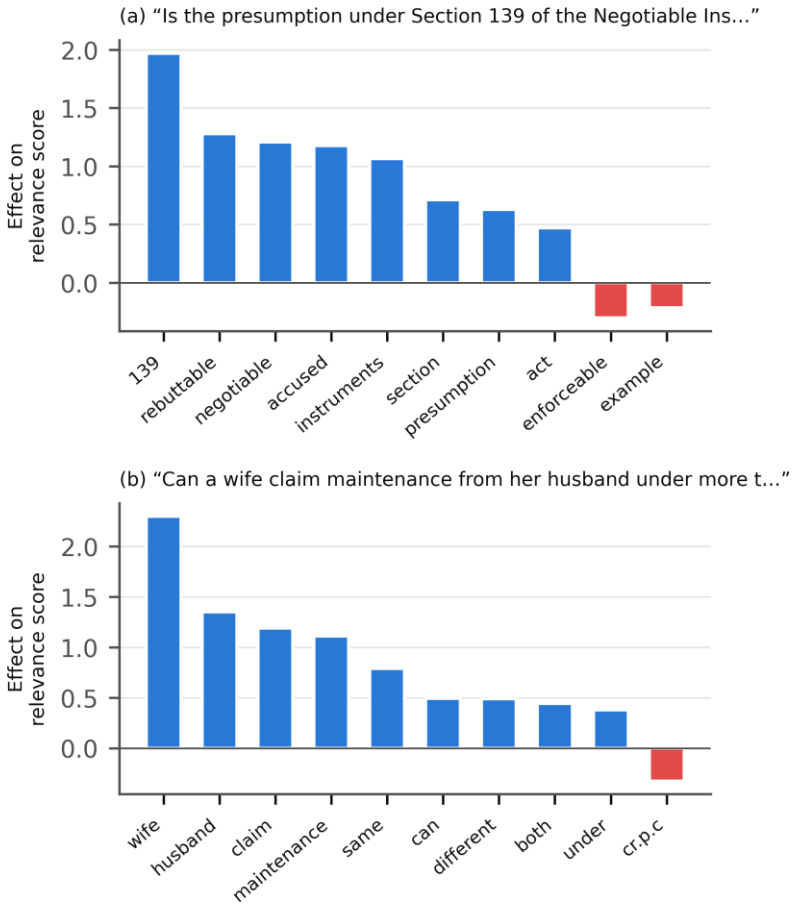


Fig. 4.21 LIME explanation of the evidence re-ranker

The linear model explains the re-ranker well, with R squared values of 0.86 and 0.75, so the explanation can be trusted. It shows that the evidence selector relies on the legally decisive words and not on incidental ones, and it lets an advocate see why a particular judgment was offered as support.

CHAPTER 5
CONCLUSION AND FUTURE WORK

DocketPilot shows that a single open-weight language model, placed inside a pipeline of retrieval, source-tagged drafting and a verifier that repairs its own output, can produce legal answers and drafts whose every citation is checked before the lawyer sees it. The full pipeline reduced unreliable citations from 16.1% for the model alone to 0.0%, the verifier detected 99.6% of faulty citations with 99.3% precision, and a verified answer cost 87.2% less than sending whole judgments to the model. Client identities never left the local machine, and the advocate approved every document before release.

The ablation study gave the clearest lesson of the project: each layer fixes a different failure. Retrieval reduces invented cases but encourages real cases to be cited for the wrong point; source tags remove invention; and only verification with automatic correction removes misattribution. Verification should therefore be treated as a core component of legal AI and not as an optional extra.

FUTURE WORK

- A study with practising advocates measuring time saved and the quality of the drafts.
- Wider High Court coverage and complete coverage of SCC and AIR citations, so that every citation format can be checked.
- Optical character recognition for scanned case papers, with the same masking review.
- Support for Tamil, Hindi and other Indian languages in the interface and in the papers.
- Hosting on a cloud server with encrypted storage and firm-level key management.
- Larger evaluation sets once higher API quotas are available.

REFERENCES

[1] LegalMind: Agentic AI-driven process optimization and cost reduction in legal services using DeepSeek, IEEE Access, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3586781.

[2] Neethi AI: An AI-powered multi-agent system for Indian legal advisory, IEEE Conference Publication, 2026.

[3] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning and D. E. Ho, "Hallucination-free? Assessing the reliability of leading AI legal research tools," Journal of Empirical Legal Studies, vol. 22, pp. 216-242, 2025.

[4] M. Dahl, V. Magesh, M. Suzgun and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," Journal of Legal Analysis, vol. 16, no. 1, pp. 64-93, 2024.

[5] P. Liu, D. Stammbach and P. Henderson, "Who checks the citations? Benchmarking legal hallucination detection," arXiv:2606.21155, 2026.

[6] Deepanshu, D. Saxena, D. Rana, A. Varshney and S. R. Laskar, "NyayaAI: An AI-powered legal assistant using multi-agent architecture and retrieval-augmented generation," arXiv:2605.10155, 2026.

[7] V. Vishnuprabha, D. M. Viswanathan, R. Rajesh and A. V. Pillai, "Augmented question-guided retrieval (AQgR) of Indian case law with LLM, RAG, and structured summaries," arXiv:2508.04710, 2025.

[8] M. Hany, M. ElSheraey, M. Said and P. Naoum, "Inject or navigate? Token-efficient retrieval for LLM analysis of transactional legal documents," arXiv:2607.05764, 2026.

[9] J. Bose, "Falkor-IRAC: Graph-constrained generation for verified legal reasoning in Indian judicial AI," arXiv:2605.14665, 2026.

[10] S. Chen, H. Yin and S. Zhou, "LegalCiteBench: Evaluating citation reliability in legal language models," in Proc. ICML, 2026.

[11] V. Ovcharov, "Citation grounding measures the oracle: Graph coverage determines reported LLM hallucination rates in law," arXiv:2606.00898, 2026.

[12] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, pp. 9459-9474.

[13] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. ACM SIGIR, 2009, pp. 758-759.

[14] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[15] S. Xiao, Z. Liu, P. Zhang and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," arXiv:2309.07597, 2023.

[16] M. T. Ribeiro, S. Singh and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in Proc. ACM SIGKDD, 2016, pp. 1135-1144.

[17] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," 2025.

[18] LangChain, "LangGraph: Building stateful, multi-actor applications with LLMs," 2025. [Online]. Available: https://langchain-ai.github.io/langgraph/

[19] M. Honnibal, I. Montani, S. Van Landeghem and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020.

[20] A. Joshi et al., "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in Proc. ACL, 2024.

[21] V. Malik et al., "ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation," in Proc. ACL-IJCNLP, 2021, pp. 4046-4062.

[22] Indian Supreme Court and High Court judgments, Registry of Open Data on AWS (CC-BY-4.0); India Code, Legislative Department, Government of India.

APPENDIX

A.1 Agent graph (LangGraph workflow) (docketpilot/agents/graph.py)

```
"""LangGraph wiring. State is checkpointed in PostgreSQL after every step, so a paused or
crashed run resumes.

orchestrator -> document_analysis -> research -> drafting -> verification -> strategy_risk ->
monitoring
  -> approval (PAUSE: lawyer approves / edits) -> [edit: back to verification] -> finalize

A draft whose verification is 'blocked' cannot be approved; amber items must each be
acknowledged.
"""
from datetime import UTC, datetime
from functools import lru_cache
from typing import Any

from langgraph.checkpoint.postgres import PostgresSaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command, interrupt
from psycopg.rows import dict_row
from psycopg_pool import ConnectionPool

from docketpilot.agents import nodes
from docketpilot.agents.nodes import State, save_draft
from docketpilot.config import get_settings


def approval(state: State) -> dict[str, Any]:
    """Pause until the lawyer decides. Resume value: {"action": "approve", "user_id",
"acknowledged": [...]}
    or {"action": "revise", "text": "..."}."""
    report = state["verification"]
    decision = interrupt({"draft_id": state["draft_id"], "release": report["release"],
                          "blocking_items": report["blocking_items"],
                          "items_needing_acknowledgement":
report["items_needing_acknowledgement"]})
    if decision.get("action") == "revise":
        return {"draft_text": decision["text"], "approval": {"action": "revise"}}
    if report["release"] == "blocked":
        return {"approval": {"action": "refused", "reason": "Verification found blocking
problems. Edit the draft."}}
    missing = set(report["items_needing_acknowledgement"]) -
set(decision.get("acknowledged", []))
    if missing:
        return {"approval": {"action": "refused", "reason": f"Acknowledge these items first:
{sorted(missing)}"}}
    return {"approval": {"action": "approve", "user_id": decision["user_id"],
                         "acknowledged": decision.get("acknowledged", [])}}
```

```
def after_approval(state: State) -> str:
    action = state.get("approval", {}).get("action")
    return {"revise": "verification", "approve": "finalize"}.get(action or "", "approval")


def finalize(state: State) -> dict[str, Any]:
    """Re-insert real names from the local vault and write the Word file (only after
approval)."""
    from docketpilot.drafting.docx_out import write_docx
    from docketpilot.privacy import redact as pii
    from docketpilot.privacy import vault

    mapping = vault.load(state["matter_id"])
    body = pii.restore(state["draft_text"], mapping)
    ack = state["approval"].get("acknowledged", [])
    path = nodes.OUTPUTS / state["matter_id"] / f"{state['draft_id']}.docx"
    write_docx(path, body, state["verification"], ack)
    save_draft(state["draft_id"], status="approved", output_file=str(path), acknowledged=ack,
               approved_by=state["approval"]["user_id"], approved_at=datetime.now(UTC),
trace=state.get("trace", []))
    return {"status": "approved"}


def _ok(next_node: str) -> Any:
    def route(state: State) -> str:
        return END if state.get("status") == "failed" else next_node
    return route


def build() -> StateGraph:
    g = StateGraph(State)
    for name in ("orchestrator", "document_analysis", "research", "drafting", "verification",
"strategy_risk",
                 "monitoring"):
        g.add_node(name, getattr(nodes, name))
    g.add_node("approval", approval)
    g.add_node("finalize", finalize)
    g.add_edge(START, "orchestrator")
    chain = ["orchestrator", "document_analysis", "research", "drafting", "verification",
"strategy_risk",
             "monitoring", "approval"]
    for a, b in zip(chain, chain[1:], strict=False):
        g.add_conditional_edges(a, _ok(b))
    g.add_conditional_edges("approval", after_approval)
    g.add_edge("finalize", END)
    return g


@lru_cache
```

```
def checkpointer() -> PostgresSaver:
   url = get_settings().database_url.replace("postgresql+asyncpg://", "postgresql://")
   pool = ConnectionPool(url, max_size=4, kwargs={"autocommit": True,
"prepare_threshold": 0, "row_factory": dict_row},
                         open=True)
   saver = PostgresSaver(pool)  # type: ignore[arg-type]
   saver.setup()
   return saver


@lru_cache
def app() -> Any:
   return build().compile(checkpointer=checkpointer())


def start(draft_id: str, matter_id: str, workflow: str, documents: list[dict[str, Any]],
          doc_type: str | None = None, instructions: str | None = None) -> dict[str, Any]:
   cfg = {"configurable": {"thread_id": draft_id}}
   init: dict[str, Any] = {"draft_id": draft_id, "matter_id": matter_id, "workflow": workflow,
"documents": documents,
                           "trace": []}
   if doc_type:
      init["doc_type"] = doc_type
   if instructions:
      init["instructions"] = instructions
   return app().invoke(init, cfg)


def resume(draft_id: str, decision: dict[str, Any]) -> dict[str, Any]:
   cfg = {"configurable": {"thread_id": draft_id}}
   return app().invoke(Command(resume=decision), cfg)


def current(draft_id: str) -> dict[str, Any]:
   snap = app().get_state({"configurable": {"thread_id": draft_id}})
   return {"values": snap.values, "next": list(snap.next),
           "interrupt": [i.value for t in snap.tasks for i in t.interrupts]}
```

A.2 Privacy gateway (masking) (docketpilot/privacy/redact.py)

```
"""PII detection and redaction.

Rule-based recognisers for Indian identifiers + spaCy (en_core_web_sm) for names,
organisations, places.
Recall matters more than precision here (spec 10.1), so rules err on the side of masking.
"""
import re
from dataclasses import dataclass, field
from functools import lru_cache
from typing import Any

# ---------- Aadhaar checksum (Verhoeff) ----------
```

```
_D = [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9], [1, 2, 3, 4, 0, 6, 7, 8, 9, 5], [2, 3, 4, 0, 1, 7, 8, 9, 5, 6],
     [3, 4, 0, 1, 2, 8, 9, 5, 6, 7], [4, 0, 1, 2, 3, 9, 5, 6, 7, 8], [5, 9, 8, 7, 6, 0, 4, 3, 2, 1],
     [6, 5, 9, 8, 7, 1, 0, 4, 3, 2], [7, 6, 5, 9, 8, 2, 1, 0, 4, 3], [8, 7, 6, 5, 9, 3, 2, 1, 0, 4],
     [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]]
_P = [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9], [1, 5, 7, 6, 2, 8, 3, 0, 9, 4], [5, 8, 0, 3, 7, 9, 6, 1, 4, 2],
     [8, 9, 1, 6, 0, 4, 3, 5, 2, 7], [9, 4, 5, 3, 1, 2, 6, 8, 7, 0], [4, 2, 8, 6, 5, 7, 3, 9, 0, 1],
     [2, 7, 9, 3, 8, 0, 6, 4, 1, 5], [7, 0, 4, 6, 9, 1, 3, 2, 5, 8]]
_INV = [0, 4, 3, 2, 1, 5, 6, 7, 8, 9]


def verhoeff_valid(num: str) -> bool:
    c = 0
    for i, ch in enumerate(reversed(num)):
        c = _D[c][_P[i % 8][int(ch)]]
    return c == 0


def verhoeff_digit(num: str) -> str:
    c = 0
    for i, ch in enumerate(reversed(num)):
        c = _D[c][_P[(i + 1) % 8][int(ch)]]
    return str(_INV[c])


# ---------- rule recognisers ----------
RULES: list[tuple[str, re.Pattern[str]]] = [
    ("EMAIL", re.compile(r"\b[\w.+-]+@[\w-]+(?:\.[\w-]+)+\b")),
    ("AADHAAR", re.compile(r"\b[2-9]\d{3}[\s-]?\d{4}[\s-]?\d{4}\b")),
    ("PAN", re.compile(r"\b[A-Z]{3}[ABCFGHLJPT][A-Z]\d{4}[A-Z]\b")),
    ("PHONE", re.compile(r"(?<![\w/])(?:\+91[\s-]?|0)?[6-9]\d{4}[\s-]?\d{5}\b")),
    ("IFSC", re.compile(r"\b[A-Z]{4}0[A-Z0-9]{6}\b")),
    ("BANK_ACCOUNT", re.compile(r"(?i)(?:a/?c|account)\s*(?:no\.?|number|#)?\s*[:.-
]?\s*(\d[\d\s-]{7,20}\d)")),
    ("VEHICLE", re.compile(r"\b[A-Z]{2}[\s-]?\d{1,2}[\s-]?[A-Z]{1,3}[\s-]?\d{4}\b")),
    ("ADDRESS", re.compile(  # "No. 21, Temple Street, Madurai - 625001"; never starts at a
cheque/case number,
        # and never runs across words that belong to a transaction ("dated", "Rs.", "cheque",
"for")
        r"(?i)(?<!cheque\s)(?<!cheque)(?<!memo\s)(?<!case\s)(?<!a/c\s)"
        r"\b(?:(?:door|house|flat|plot)\s*no\.?|no\.)\s*[\w/-]{1,8},?\s+"
        r"(?:(?!\b(?:dated|rs|cheque|for|amount|returned)\b)[^\n]){3,100}?(?:-|\s)\s*\d{6}\b")),
    ("PERSON", re.compile(  # honorifics and relation markers common in Indian legal
documents
        r"\b(?:Mr|Mrs|Ms|Smt|Shri|Sri|Thiru|Tmt|Dr|Kumari|Selvi)\.?\s+((?:[A-Z]\.\s?)*[A-Z][a-
z]+(?:\s+[A-Z][a-z]+){0,3})"
        r"|\b(?:[SDW]/o|son of|daughter of|wife of)\s+((?:[A-Z]\.\s?)*[A-Z][a-z]+(?:\s+[A-Z][a-
z]+){0,3})"

r"|\b(?:complainant|accused|drawer|payee|petitioner|respondent|proprietor|director|partner)\s+
"
        r"((?:[A-Z]\.\s?)*[A-Z][a-z]+(?:\s+[A-Z][a-z]+){1,3})")),
```

```
    ("ORG", re.compile(  # common Indian business-name endings
        r"\b((?:M/s\.?\s+)?(?:[A-Z][\w&'.-
]*\s+){1,5}(?:Traders|Textiles|Agencies|Agency|Exports|Imports|Stores|"

r"Enterprises|Industries|Company|Corporation|Associates|Finance|Finances|Motors|Builders|
Hardware|"

r"Constructions|Mills|Jewellers|Pharma|Foods)(?:\s+(?:Pvt\.?|Private))?(?:\s+(?:Ltd\.?|Limite
d|LLP))?"
        r"|(?:M/s\.?\s+)?(?:[A-Z][\w&'.-]*\s+){1,5}(?:Pvt\.?|Private)\s+(?:Ltd\.?|Limited)|"
        r"(?:[A-Z][\w&'.-]*\s+){1,4}&\s+Co\.?)")),
]
NER_LABELS = {"PERSON": "PERSON", "ORG": "ORG", "GPE": "PLACE", "LOC":
"PLACE", "FAC": "PLACE"}
# Never mask the law itself or courts: the model needs them, and they are public.
KEEP =
re.compile(r"(?i)\b(act|code|court|section|sanhita|adhiniyam|constitution|tribunal|magistrate|"
                r"state of|union of india|reserve bank|supreme|high court|cheque|rupees|rs)\b")


@dataclass
class Entity:
    type: str
    start: int
    end: int
    value: str
    source: str  # rule | ner | user
    placeholder: str = ""


@dataclass
class Redaction:
    text: str
    entities: list[Entity]
    mapping: dict[str, str] = field(default_factory=dict)  # placeholder -> real value


@lru_cache
def _nlp() -> Any:
    import spacy

    return spacy.load("en_core_web_sm", disable=["lemmatizer"])


def detect(text: str, use_ner: bool = True) -> list[Entity]:
    found: list[Entity] = []
    for typ, pat in RULES:
        for m in pat.finditer(text):
            if typ == "AADHAAR" and not verhoeff_valid(re.sub(r"\D", "", m.group(0))):
                continue
            g = next((i for i in range(1, (m.lastindex or 0) + 1) if m.group(i)), 0)
```

```
            found.append(Entity(typ, m.start(g), m.end(g), m.group(g), "rule"))
    if use_ner:
        for ent in _nlp()(text).ents:
            label = NER_LABELS.get(ent.label_)
            if label and len(ent.text.strip()) > 2 and not KEEP.search(ent.text):
                found.append(Entity(label, ent.start_char, ent.end_char, ent.text, "ner"))
    # Propagate: a value found once is masked everywhere it occurs (NER often spots a name only once).
    seen = {(e.type, e.value.strip()) for e in found if len(e.value.strip()) >= 4}
    for typ, value in seen:
        for m in re.finditer(rf"(?<!\w){re.escape(value)}(?!\w)", text):
            found.append(Entity(typ, m.start(), m.end(), value, "propagated"))
    # keep the longest non-overlapping spans; rules win ties
    found.sort(key=lambda e: (e.start, -(e.end - e.start), e.source != "rule"))
    out: list[Entity] = []
    for e in found:
        if out and e.start < out[-1].end:
            continue
        out.append(e)
    return out


def redact(text: str, mapping: dict[str, str] | None = None, extra: list[tuple[str, str]] | None = None,
           unmask: set[str] | None = None, use_ner: bool = True) -> Redaction:
    """mapping: existing placeholder->value map for this matter (keeps PERSON_1 the same across documents).
    extra: (type, value) pairs the lawyer added in the preview; unmask: values the lawyer said are not PII."""
    mapping = dict(mapping or {})
    reverse = {v.lower(): k for k, v in mapping.items()}
    entities = [e for e in detect(text, use_ner) if not (unmask and e.value in unmask)]
    for typ, value in extra or []:
        for m in re.finditer(re.escape(value), text):
            if not any(m.start() < e.end and e.start < m.end() for e in entities):
                entities.append(Entity(typ, m.start(), m.end(), value, "user"))
    entities.sort(key=lambda e: e.start)
    counters: dict[str, int] = {}
    for k in mapping:
        t, _, n = k.rpartition("_")
        if n.isdigit():
            counters[t] = max(counters.get(t, 0), int(n))
    parts, pos = [], 0
    for e in entities:
        key = e.value.strip().lower()
        ph = reverse.get(key)
        if ph is None:
            counters[e.type] = counters.get(e.type, 0) + 1
            ph = f"{e.type}_{counters[e.type]}"
            mapping[ph] = e.value.strip()
```

```
            reverse[key] = ph
        e.placeholder = ph
        parts.append(text[pos:e.start])
        parts.append(ph)
        pos = e.end
    parts.append(text[pos:])
    return Redaction("".join(parts), entities, mapping)


def restore(text: str, mapping: dict[str, str]) -> str:
    """Put real values back (only after the lawyer approves the draft). Longest placeholders
first so
    PERSON_12 is not broken by PERSON_1."""
    for ph in sorted(mapping, key=len, reverse=True):
        text = re.sub(rf"\b{re.escape(ph)}\b", mapping[ph].replace("\\", "\\\\"), text)  # literal
replacement
    return text
```

A.3 Encrypted placeholder vault (docketpilot/privacy/vault.py)

```
"""Local encrypted vault: placeholder -> real value, one file per matter. Never sent anywhere,
never logged."""
import json
from pathlib import Path

from cryptography.fernet import Fernet, InvalidToken

from docketpilot.config import get_settings

VAULT_DIR = Path(__file__).resolve().parents[2] / "vault"


class VaultError(Exception):
    pass


def _fernet() -> Fernet:
    key = get_settings().vault_key
    if key is None:
        raise VaultError("VAULT_KEY is not set")
    return Fernet(key.get_secret_value().encode())


def _path(matter_id: str) -> Path:
    safe = "".join(ch for ch in matter_id if ch.isalnum() or ch in "-_")
    if not safe:
        raise VaultError("bad matter id")
    return VAULT_DIR / f"{safe}.vault"


def load(matter_id: str) -> dict[str, str]:
    p = _path(matter_id)
```

```
    if not p.exists():
        return {}
    try:
        return json.loads(_fernet().decrypt(p.read_bytes()))
    except InvalidToken as e:
        raise VaultError("vault cannot be decrypted with the current key") from e


def save(matter_id: str, mapping: dict[str, str]) -> None:
    VAULT_DIR.mkdir(exist_ok=True)
    _path(matter_id).write_bytes(_fernet().encrypt(json.dumps(mapping,
ensure_ascii=False).encode()))


def delete(matter_id: str) -> None:
    _path(matter_id).unlink(missing_ok=True)
```

A.4 Automatic correction of citations (docketpilot/verification/correct.py)

```
"""Auto-correction after verification: the user should not have to deal with bad citations.

  * wrong citation for a case that exists   -> replaced with the correct citation of that case
  * case not found / does not support the point / quote not in the judgment -> citation
removed, and a
    sentence left with no remaining authority is removed too (an unsupported legal claim is not
kept)
  * everything else stays as it is

Returns the corrected text, a short list of what changed, and a report that matches the
corrected text.
"""
import re
from typing import Any

import psycopg

from docketpilot.knowledge.citations import find_citations
from docketpilot.verification.engine import resolve
from docketpilot.verification.extract import CASE_NAME, CitedItem, _clean_name,
_sentence_bounds

BAD_PROPOSITION = {"contradicts", "does_not_support", "unsupported"}
BAD_QUOTE = {"not_found", "only_in_headnote"}


def _display_citation(conn: psycopg.Connection, case_id: str) -> str | None:
    with conn.cursor() as cur:
        cur.execute("""SELECT raw FROM case_citations WHERE case_id=%s
                    ORDER BY CASE kind WHEN 'neutral' THEN 0 WHEN 'scr' THEN 1
WHEN 'hc_neutral' THEN 2 ELSE 3 END,
                             confidence DESC LIMIT 1""", (case_id,))
        r = cur.fetchone()
```

```
    return r[0] if r else None


def _short_name(title: str) -> str:
    title = re.sub(r"\s+", " ", title).strip()
    m = re.search(r"\bof\s+(.*?)\s+Vs\.?\s+(.*)$", title, re.I) or
re.search(r"^(.*?)\s+versus\s+(.*)$", title, re.I)
    name = f"{m.group(1).strip().title()} v. {m.group(2).strip().title()}" if m else title.title()
    return re.sub(r"\s+(And|&)\s+(Another|Others|Ors\.?|Anr\.?)\b.*$", "", name, flags=re.I)


def _fix_for(conn: psycopg.Connection, c: dict[str, Any]) -> tuple[str, str | None, str]:
    """(action, replacement, reason) for one citation result. action: keep | replace | remove."""
    prop = (c.get("proposition") or {}).get("label")
    if any(q["status"] in BAD_QUOTE for q in c.get("quotes", [])):
        return "remove", None, "the quoted words are not in that judgment"
    if prop in BAD_PROPOSITION:
        return "remove", None, "that judgment does not support the point"
    if c["verdict"] == "NOT_FOUND":
        return "remove", None, "the case could not be found"
    if c["verdict"] == "CONFLICT":
        if c.get("case_name"):  # the named case may exist under its correct citation
            cands = resolve(conn, CitedItem("x", c["case_name"], c["case_name"], None,
c.get("year"), 0, 0, ""))
            if cands:
                cit = _display_citation(conn, cands[0].case_id)
                if cit:
                    return "replace", f"{c['case_name']}, {cit}", "the citation pointed to a different
case"
        return "remove", None, "the citation pointed to a different case"
    if c["verdict"] == "PARTIAL" and c.get("candidates"):
        best = c["candidates"][0]
        if best.get("via") == "name_year":  # names match a real case, citation string wrong
            cit = _display_citation(conn, best["case_id"])
            if cit:
                name = c.get("case_name") or _short_name(best["title"])
                return "replace", f"{name}, {cit}", "wrong year or page in the citation"
    return "keep", None, ""


def _cleanup(text: str) -> str:
    text = re.sub(r"\(\s*[;,]?\s*\)", "", text)          # empty brackets
    text = re.sub(r"\(\s*[;,]\s*", "(", text)
    text = re.sub(r"\s*[;,]\s*\)", ")", text)
    text = re.sub(r";\s*;", ";", text)
    text = re.sub(r"[ \t]+([.,;:])", r"\1", text)
    text = re.sub(r"[ \t]{2,}", " ", text)
    text = re.sub(r"^[ \t]*[-•*][ \t]*$\n?", "", text, flags=re.M)  # bullets left empty
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()
```

```
def autocorrect(conn: psycopg.Connection, text: str, report: dict[str, Any]) -> tuple[str,
list[dict[str, Any]], dict[str, Any]]:
    cits = report.get("citations", [])
    plans = [(c, *_fix_for(conn, c)) for c in cits]
    removed = [c for c, a, _, _ in plans if a == "remove"]
    kept_spans = [(c["start"], c["end"]) for c, a, _, _ in plans if a != "remove"]

    # Sentences whose every citation is removed lose their claim too.
    drop: list[tuple[int, int]] = []
    for c in removed:
        s, e = _sentence_bounds(text, c["start"], c["end"])
        if not any(s <= ks < e for ks, _ in kept_spans) and (s, e) not in drop:
            drop.append((s, e))

    edits: list[tuple[int, int, str]] = [(s, e, "") for s, e in drop]
    for c, action, repl, _ in plans:
        inside_dropped = any(s <= c["start"] < e for s, e in drop)
        if action == "remove" and not inside_dropped:
            edits.append((c["start"], c["end"], ""))
        elif action == "replace" and not inside_dropped:
            edits.append((c["start"], c["end"], repl or ""))
    # apply from the end so earlier offsets stay valid; skip overlaps
    out, last_start = text, len(text) + 1
    for s, e, new in sorted(edits, key=lambda x: x[0], reverse=True):
        if e > last_start:
            continue
        out = out[:s] + new + out[e:]
        last_start = s
    out = _cleanup(out)

    changes = [{"action": a, "original": c["raw"], "replacement": r, "reason": why}
               for c, a, r, why in plans if a != "keep"]
    # Names of cases the model mentioned without any citation could not be checked: take
them out too.
```